

Data Visualization & Analysis Lab

Lab Report 1: NumPy, Pandas, and Matplotlib Basics

Name: Aastha Baheti

SAP ID: 590012973

Batch: B2 Data Science

Section 1

1. Creating and Manipulating NumPy Arrays

Code:

```
import numpy as np

# Creating and manipulating NumPy arrays
arr1 = np.array([1, 2, 3, 4])
print(arr1)

arr2 = np.zeros((2, 3))
print(arr2)

arr3 = np.ones((3, 3))
print(arr3)

arr4 = np.arange(0, 10, 2)
print(arr4)

arr5 = np.linspace(0, 1, 5)
print(arr5)
```

Output:

```
[1 2 3 4]
[[0. 0. 0.]
 [0. 0. 0.]]
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

```
[0 2 4 6 8]
[0.  0.25 0.5  0.75 1.  ]
```

2. Random Number Generation and Sampling

Code:

```
import numpy as np

# Random number generation and sampling
rand1 = np.random.rand(3, 3) # Uniform distribution
print(rand1)

rand2 = np.random.randn(3, 3) # Normal distribution
print(rand2)

rand3 = np.random.randint(1, 10, 5) # Random integers
print(rand3)
```

Output:

```
[[0.83024108 0.41831635 0.36260446]
 [0.07033123 0.96333599 0.38034897]
 [0.76987615 0.49349978 0.88337605]]
[[-0.64967962  0.30800243 -0.44056158]
 [ 0.24020083 -0.93916759  0.92015744]
 [ 1.84543745 -1.05557419  1.00044403]]
[1 2 3 7 6]
```

3. Indexing, Slicing, and Selection Techniques

Code:

```
import numpy as np

# Indexing, slicing, and selection techniques
arr = np.array([10, 20, 30, 40, 50])

# Indexing
print(arr[0])

# Slicing
print(arr[1:4])
```

```
# Boolean indexing
print(arr[arr > 25])

# 2D Array
arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr2d[1, 2])
```

Output:

```
10
[20 30 40]
[30 40 50]
6
```

4. Basic Array Operations and Mathematical Functions

Code:

```
import numpy as np

# Basic array operations and mathematical functions
arr = np.array([1, 2, 3, 4])
print(arr + 2)
print(arr * 2)

# Mathematical functions
print(np.sqrt(arr))
print(np.exp(arr))
print(np.mean(arr))
print(np.sum(arr))
print(np.max(arr))
```

Output:

```
[3 4 5 6]
[2 4 6 8]
[1.         1.41421356 1.73205081 2.         ]
[ 2.71828183  7.3890561  20.08553692 54.59815003]
2.5
10
4
```

Section 2:

This section explores data structures (Series and DataFrames), handling missing data (cleaning NaN entries), combining datasets (merging, joining, grouping, concatenating), and reading/writing real datasets (using the `iris.csv` database).

1. Creating and Working with Series and DataFrames

Code:

```
import pandas as pd

# Creating a Series
s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])
print("Series:")
print(s)
print("Accessing elements by label:", s['c'])
print("Accessing elements by position:", s.iloc[2])
print()

# Creating a DataFrame from a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}
df = pd.DataFrame(data)
print("DataFrame:")
print(df)
print("\nAccessing columns:")
print(df['Name'])
print("\nAccessing rows using loc:")
print(df.loc[1])
```

Output:

```
Series:
a      10
b      20
c      30
d      40
e      50
dtype: int64
Accessing elements by label: 30
```

Accessing elements by position: 30

DataFrame:

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago
3	David	40	Houston

Accessing columns:

```
0    Alice
1     Bob
2  Charlie
3    David
```

Name: Name, dtype: object

Accessing rows using loc:

```
Name      Bob
Age       30
City  Los Angeles
Name: 1, dtype: object
```

2. Handling Missing or Null Data

Code:

```
import pandas as pd
import numpy as np

data_with_nan = {
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, np.nan, 8],
    'C': [9, 10, 11, 12]
}
df_nan = pd.DataFrame(data_with_nan)
print("DataFrame with NaN:")
print(df_nan)

# Check for null values
print("\nNull check (isna):")
print(df_nan.isna())

# Drop missing values
```

```

print("\nDropping rows with missing values:")
print(df_nan.dropna())

# Fill missing values
print("\nFilling missing values with 0:")
print(df_nan.fillna(value=0))

# Fill missing values with column mean
print("\nFilling missing values with mean of column A:")
print(df_nan['A'].fillna(value=df_nan['A'].mean()))

```

Output:

DataFrame with NaN:

	A	B	C
0	1.0	5.0	9
1	2.0	NaN	10
2	NaN	NaN	11
3	4.0	8.0	12

Null check (isna):

	A	B	C
0	False	False	False
1	False	True	False
2	True	True	False
3	False	False	False

Dropping rows with missing values:

	A	B	C
0	1.0	5.0	9
3	4.0	8.0	12

Filling missing values with 0:

	A	B	C
0	1.0	5.0	9
1	2.0	0.0	10
2	0.0	0.0	11
3	4.0	8.0	12

Filling missing values with mean of column A:

0	1.000000
1	2.000000
2	2.333333

```
3      4.000000
Name: A, dtype: float64
```

3. Using Groupby, Merging, Joining, and Concatenation

Code:

```
import pandas as pd

# Groupby
data_group = {
    'Company': ['GOOG', 'GOOG', 'MSFT', 'MSFT', 'FB', 'FB'],
    'Person': ['Sam', 'Charlie', 'Amy', 'Vanessa', 'Carl', 'Sarah'],
    'Sales': [200, 120, 340, 124, 243, 350]
}
df_group = pd.DataFrame(data_group)
print("DataFrame for Groupby:")
print(df_group)
print("\nMean sales by company:")
print(df_group.groupby('Company')['Sales'].mean())
print()

# Merging (SQL-like join)
df1 = pd.DataFrame({'Key': ['K0', 'K1', 'K2', 'K3'], 'A': ['A0', 'A1', 'A2', 'A3']})
df2 = pd.DataFrame({'Key': ['K0', 'K1', 'K2', 'K3'], 'B': ['B0', 'B1', 'B2', 'B3']})
print("df1:")
print(df1)
print("df2:")
print(df2)
print("\nMerged DataFrame:")
print(pd.merge(df1, df2, on='Key'))
print()

# Joining (based on index)
left = pd.DataFrame({'A': ['A0', 'A1', 'A2']}, index=['K0', 'K1', 'K2'])
right = pd.DataFrame({'B': ['B0', 'B1', 'B2']}, index=['K0', 'K2', 'K3'])
print("Left (index K0, K1, K2):")
print(left)
print("Right (index K0, K2, K3):")
print(right)
print("\nJoined DataFrame (left join by default):")
```

```

print(left.join(right))
print()

# Concatenation
df3 = pd.DataFrame({'A': ['A0', 'A1'], 'B': ['B0', 'B1']}, index=[0, 1])
df4 = pd.DataFrame({'A': ['A2', 'A3'], 'B': ['B2', 'B3']}, index=[2, 3])
print("df3:")
print(df3)
print("df4:")
print(df4)
print("\nConcatenated DataFrame (along rows):")
print(pd.concat([df3, df4]))

```

Output:

DataFrame for Groupby:

	Company	Person	Sales
0	GOOG	Sam	200
1	GOOG	Charlie	120
2	MSFT	Amy	340
3	MSFT	Vanessa	124
4	FB	Carl	243
5	FB	Sarah	350

Mean sales by company:

Company

FB 296.5

GOOG 160.0

MSFT 232.0

Name: Sales, dtype: float64

df1:

	Key	A
0	K0	A0
1	K1	A1
2	K2	A2
3	K3	A3

df2:

	Key	B
0	K0	B0
1	K1	B1
2	K2	B2
3	K3	B3

Merged DataFrame:

	Key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2
3	K3	A3	B3

Left (index K0, K1, K2):

	A
K0	A0
K1	A1
K2	A2

Right (index K0, K2, K3):

	B
K0	B0
K2	B2
K3	B3

Joined DataFrame (left join by default):

	A	B
K0	A0	B0
K1	A1	NaN
K2	A2	B2

df3:

	A	B
0	A0	B0
1	A1	B1

df4:

	A	B
2	A2	B2
3	A3	B3

Concatenated DataFrame (along rows):

	A	B
0	A0	B0
1	A1	B1
2	A2	B2
3	A3	B3

4. Reading/Writing Files and Performing Data Operations

Code:

```
import pandas as pd
import os

# Locate iris.csv in the workspace
csv_path = 'iris.csv'
possible_paths = ['iris.csv', '../iris.csv', '../../iris.csv', 'D:/5 sem/DVI
LAB/iris.csv']
for path in possible_paths:
    if os.path.exists(path):
        csv_path = path
        break

print(f"Reading dataset from: {csv_path}")
df_iris = pd.read_csv(csv_path)

# Display first few rows
print("\nFirst 5 rows of Iris dataset:")
print(df_iris.head())

# Performing basic operations
print("\nBasic data operations (descriptive statistics for Iris):")
print(df_iris.describe())

# Filtering rows
print("\nFiltering rows (Species == 'setosa' and Sepal.Length > 5.0):")
filtered_df = df_iris[(df_iris['Species'] == 'setosa') &
(df_iris['Sepal.Length'] > 5.0)]
print(filtered_df.head())

# Writing the filtered results to a new file
filtered_output = 'filtered_iris.csv'
filtered_df.to_csv(filtered_output, index=False)
print(f"\nSaved filtered dataset to '{filtered_output}'")
```

Output:

Reading dataset from: iris.csv

First 5 rows of Iris dataset:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa

```
3          4.6          3.1          1.5          0.2  setosa
4          5.0          3.6          1.4          0.2  setosa
```

Basic data operations (descriptive statistics for Iris):

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Filtering rows (Species == 'setosa' and Sepal.Length > 5.0):

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
5	5.4	3.9	1.7	0.4	setosa
10	5.4	3.7	1.5	0.2	setosa
14	5.8	4.0	1.2	0.2	setosa
15	5.7	4.4	1.5	0.4	setosa

Saved filtered dataset to 'filtered_iris.csv'

Section 3:

This section covers plotting various types of graphs, layout management, and chart customization (titles, axes labels, legends, and colors).

1. Creating and Saving Core Charts (Line, Bar, Histogram, Scatter, Box, Pie)

Code:

```
import matplotlib.pyplot as plt
import numpy as np

# Use a style for better aesthetics
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
plt.style.available else 'default')

# 1. Line Chart
x = np.linspace(0, 10, 100)
```

```
y = np.sin(x)
plt.figure(figsize=(6, 4))
plt.plot(x, y, label='Sine Wave', color='blue', linestyle='--', linewidth=2)
plt.title('Line Chart (Sine Wave)')
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.legend()
plt.tight_layout()
plt.savefig('line_chart.png')
plt.close()
```

2. Bar Chart

```
categories = ['A', 'B', 'C', 'D']
values = [15, 24, 12, 8]
plt.figure(figsize=(6, 4))
plt.bar(categories, values, color='skyblue', edgecolor='black')
plt.title('Bar Chart')
plt.xlabel('Categories')
plt.ylabel('Values')
plt.tight_layout()
plt.savefig('bar_chart.png')
plt.close()
```

3. Histogram

```
data = np.random.randn(1000)
plt.figure(figsize=(6, 4))
plt.hist(data, bins=30, color='lightgreen', edgecolor='black', alpha=0.7)
plt.title('Histogram (Normal Distribution)')
plt.xlabel('Bins')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('histogram.png')
plt.close()
```

4. Scatter Plot

```
x_scatter = np.random.rand(50)
y_scatter = np.random.rand(50)
colors = np.random.rand(50)
sizes = 100 * np.random.rand(50)
plt.figure(figsize=(6, 4))
plt.scatter(x_scatter, y_scatter, c=colors, s=sizes, alpha=0.6,
            cmap='viridis')
plt.title('Scatter Plot')
```

```

plt.xlabel('X')
plt.ylabel('Y')
plt.colorbar(label='Color intensity')
plt.tight_layout()
plt.savefig('scatter_plot.png')
plt.close()

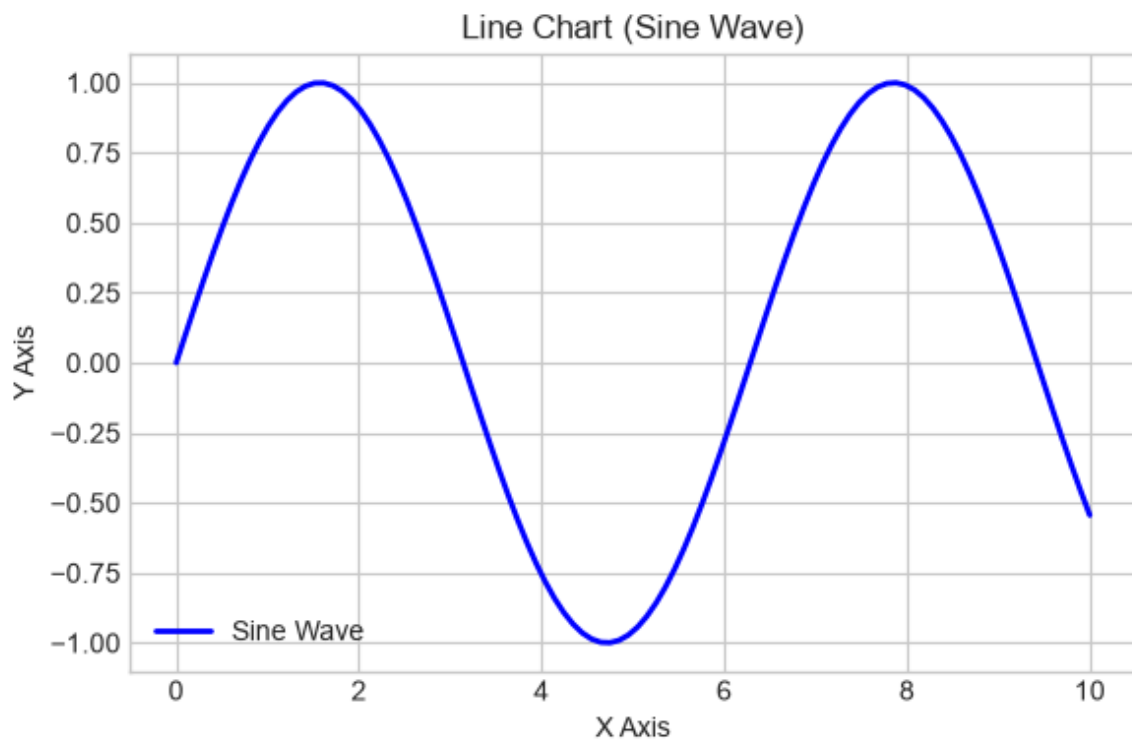
# 5. Box Plot
box_data = [np.random.normal(0, std, 100) for std in range(1, 4)]
plt.figure(figsize=(6, 4))
plt.boxplot(box_data, tick_labels=['Group 1', 'Group 2', 'Group 3'],
            patch_artist=True)
plt.title('Box Plot')
plt.xlabel('Groups')
plt.ylabel('Values')
plt.tight_layout()
plt.savefig('box_plot.png')
plt.close()

# 6. Pie Chart
labels = ['Frogs', 'Hogs', 'Dogs', 'Logs']
sizes_pie = [15, 30, 45, 10]
explode = (0, 0.1, 0, 0)
plt.figure(figsize=(6, 6))
plt.pie(sizes_pie, explode=explode, labels=labels, autopct='%1.1f%%',
        shadow=True, startangle=90)
plt.title('Pie Chart')
plt.tight_layout()
plt.savefig('pie_chart.png')
plt.close()

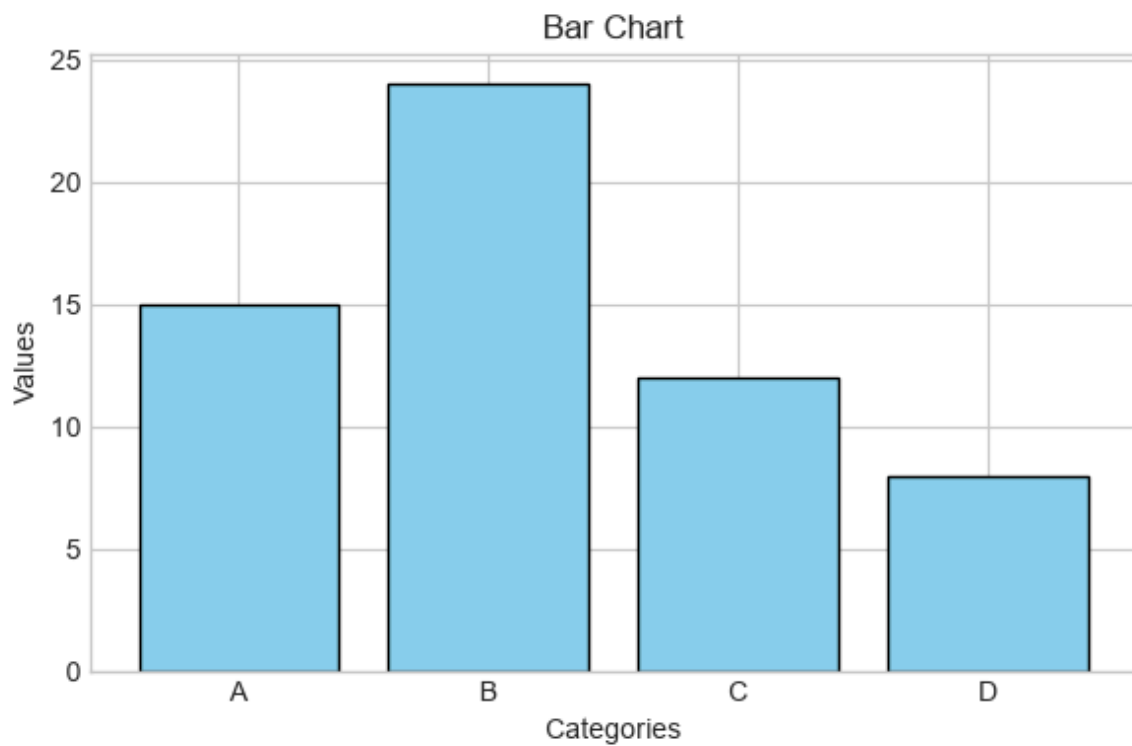
```

Visualizations:

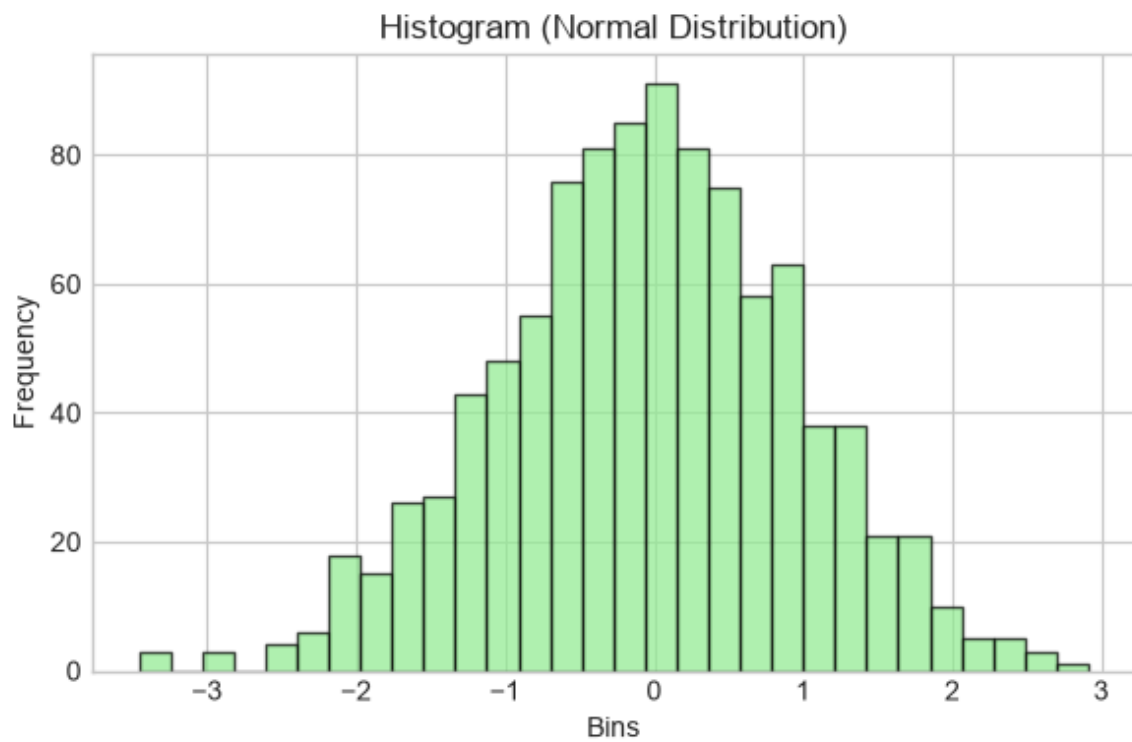
1. Line Chart



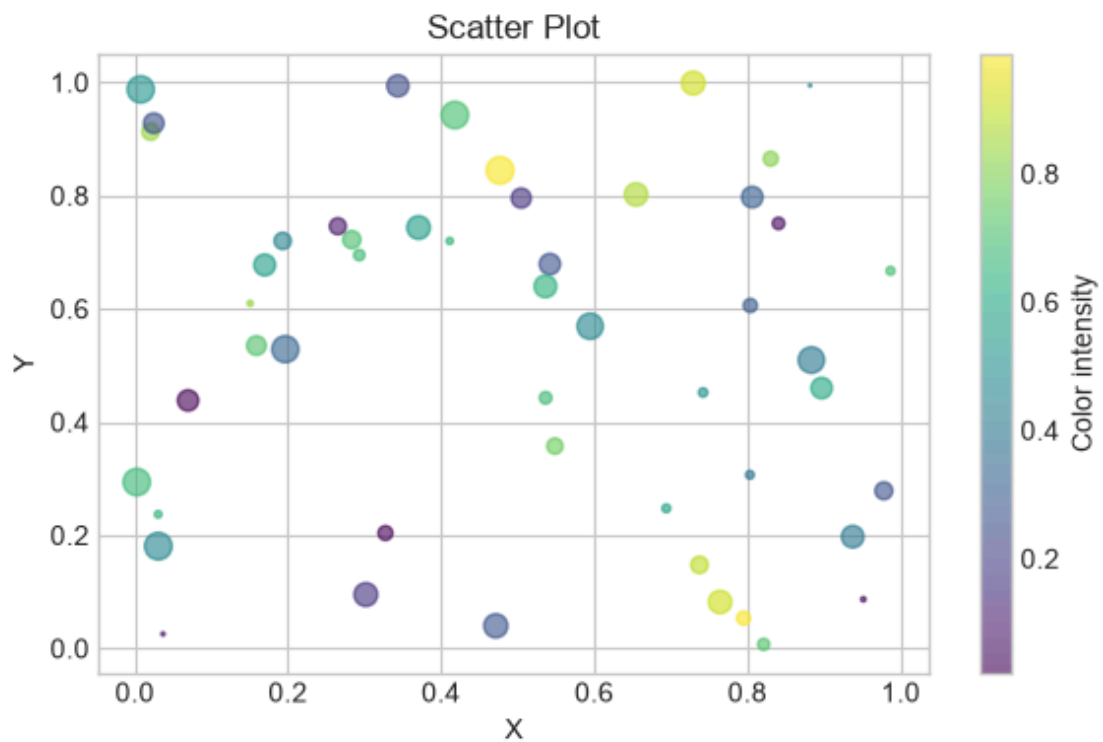
2. Bar Chart



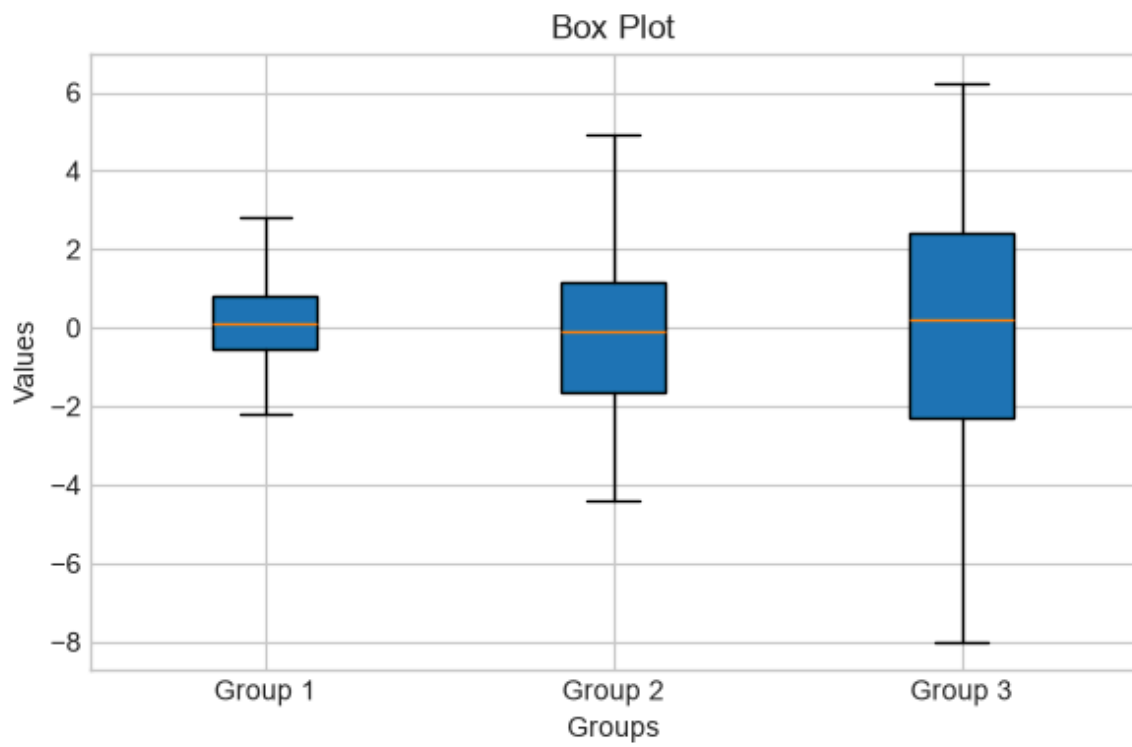
3. Histogram



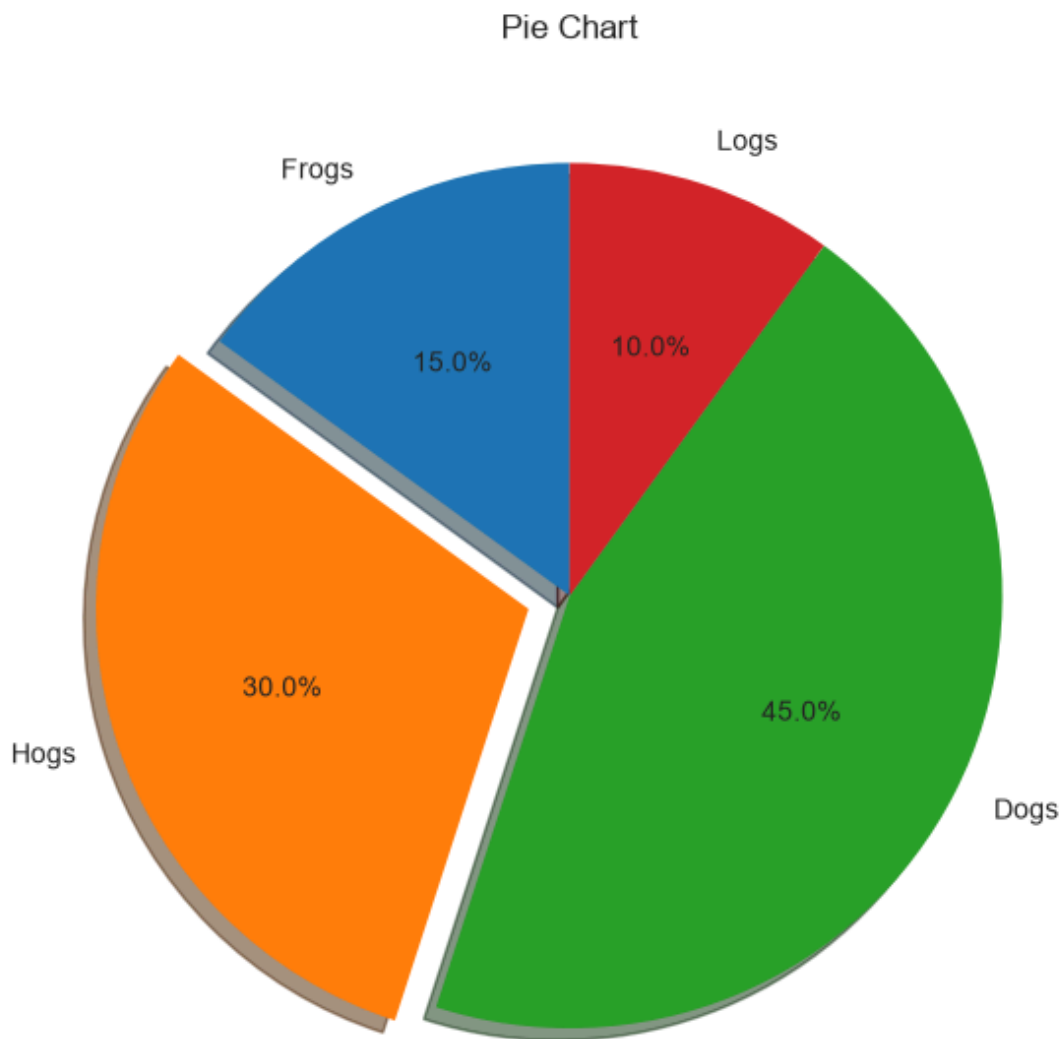
4. Scatter Plot



5. Box Plot



6. Pie Chart



2. Customizing Plots, Subplots, and Layout Management

Code:

```
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(2, 2, figsize=(10, 8))

# Subplot 1: Custom Line Plot
axes[0, 0].plot(x, np.sin(x), color='purple', linestyle='--',
label='sin(x)')
axes[0, 0].plot(x, np.cos(x), color='orange', linestyle=':', label='cos(x)')
axes[0, 0].set_title('Trigonometric Functions', fontsize=12,
fontweight='bold')
```

```

axes[0, 0].set_xlabel('Angle (radians)')
axes[0, 0].set_ylabel('Amplitude')
axes[0, 0].legend(loc='upper right')

# Subplot 2: Custom Bar Plot (Horizontal)
axes[0, 1].barh(categories, values, color=
['#ff9999', '#66b3ff', '#99ff99', '#ffcc99'])
axes[0, 1].set_title('Horizontal Bar Chart', fontsize=12, fontweight='bold')
axes[0, 1].set_xlabel('Score')
axes[0, 1].set_ylabel('Categories')

# Subplot 3: Custom Scatter with text labels
axes[1, 0].scatter(x_scatter[:10], y_scatter[:10], color='red', marker='x',
s=80)
for i in range(10):
    axes[1, 0].text(x_scatter[i], y_scatter[i], f'P{i}', fontsize=8,
ha='right')
axes[1, 0].set_title('Annotated Scatter', fontsize=12, fontweight='bold')
axes[1, 0].set_xlabel('X')
axes[1, 0].set_ylabel('Y')

# Subplot 4: Cumulative Sum Plot (NumPy Fallback)
cumulative_data = np.random.randn(100, 2).cumsum(axis=0)
axes[1, 1].plot(cumulative_data[:, 0], color='green', label='A')
axes[1, 1].plot(cumulative_data[:, 1], color='brown', label='B')
axes[1, 1].legend()
axes[1, 1].set_title('Cumulative Sum Plot', fontsize=12, fontweight='bold')
axes[1, 1].set_xlabel('Steps')
axes[1, 1].set_ylabel('Value')

# Adjust layout automatically to prevent overlap
plt.tight_layout()
plt.savefig('combined_subplots.png')
plt.close()

```

Visualizations:

Combined Subplots Grid

